A Research-Oriented Framework for Evaluating Synthetic NLP Datasets Using Fidelity, Diversity, and Downstream Utility

Raunak Singh Baweja and Om Nikam
NLP Evaluation Research Project

***Abstract*—Synthetic text is now easy to generate, but it is much harder to tell whether a synthetic dataset is actually a good stand-in for the real one. We built a tool that compares an original NLP dataset with a synthetic one along three separate lines: fidelity, diversity, and downstream utility. Fidelity covers dataset statistics, text length, vocabulary and n-gram overlap, TF-IDF similarity, semantic similarity, and class balance. Diversity looks at unique-text ratio, duplicate rate, vocabulary richness, and repeated phrases. For utility, we train the same classifier once on real data and once on synthetic data and test both on the same held-out real test set. The tool is a full-stack web application (React/TypeScript frontend, FastAPI backend, PostgreSQL storage, and a Python/scikit-learn evaluation engine with sentence-embedding support). We tested it on a Hindi three-class sentiment dataset with 2,479 original and 2,480 synthetic records. The composite score was 53/100: fidelity 41%, diversity 22%, and utility 100% (raw macro-F1 retention of about 120%). The synthetic data matched the class balance and trained a good classifier, yet it shared very little vocabulary with the original and 63% of its rows were duplicates. Accuracy alone would have hidden both problems.**

***Index Terms*—Synthetic data, NLP, dataset evaluation, text classification, fidelity, diversity, utility, sentiment analysis, TF-IDF, semantic similarity.**

I. Introduction

Synthetic data is used to enlarge training sets, cut annotation cost, and avoid sharing private records. In NLP, though, a synthetic corpus can look fine at a glance and still miss the word patterns and structure of the real data. It can also go the other way: a model trained on it scores well, but the text itself is repetitive or copied too closely from the source. One number cannot show all of this, so each kind of quality needs to be measured on its own.

Large pretrained language models made synthetic text much easier to produce. Transformers [1] and encoders like BERT [2] gave everyone strong text representations, and few-shot models [3] can now write a whole labeled dataset from a few prompts. Generated training data is now common for classification and instruction tuning [4–6]. Earlier augmentation methods such as synonym replacement and back-translation [7, 8] showed that artificial examples can improve classifier accuracy, especially in low-resource settings.

So what makes a synthetic dataset "good"? If it copies the original too closely, it adds nothing new and may even leak real training records [9]. If it gives high accuracy, it may still be built from a handful of templates. Havrilla *et al.* point out that quality, diversity, and downstream effects interact in ways that are not obvious [10], and Yu *et al.* found that generated corpora often lose diversity and carry bias [11].

Hindi adds its own difficulties. Most evaluation tools and embedding models are built mainly for English and a few other high-resource languages [12]. Hindi uses the Devanagari script, has rich morphology, and is often mixed with English, so tokenization and word counts behave differently. IndicNLPSuite has helped [13], and Doshi and Bhattacharyya note that bad generated examples in multilingual data are hard to spot [14]. We therefore wanted one pipeline that runs both datasets through the same Unicode-aware preprocessing and reports every metric with its definition.

Our research question is simple: how closely does a synthetic dataset match an original one, both in what it looks like and in how useful it is? We split this into three parts. (RQ1) Does the synthetic corpus keep the lexical, distributional, and semantic properties of the original? (RQ2) Does it have enough variety, or is it full of duplicates and repeated phrases? (RQ3) Can a classifier trained only on synthetic data do well on real held-out data? The tool answers each part separately and also gives an overall score whose weights are stated openly.

The main contributions of this paper are as follows:

- A three-dimensional evaluation formulation that separates fidelity, diversity, and downstream utility, with every component metric defined by an explicit formula.
- A controlled Synthetic→Real versus Real→Real utility protocol in which both models are evaluated on the identical held-out original test split.
- A full-stack, reproducible implementation with dataset validation, aligned metric computation, experiment history, and CSV/JSON/PDF reporting.
- An experimental evaluation on a Hindi three-class sentiment corpus that demonstrates how high utility can coexist with poor lexical fidelity and diversity.

II. Related Work

A. Synthetic Data Generation for NLP

Synthetic text has been produced by rule-based augmentation, paraphrasing, and prompting of pretrained language models. Wei and Zou proposed simple edit-based operations for small classification datasets [7], and Feng *et al.* surveyed NLP augmentation more broadly [8]. Larger generators have been used to build complete datasets for zero-shot learning [4, 5] and to bootstrap instruction data [6]. These works concentrate on generation, and

evaluation is usually limited to the accuracy of the target task.

B. Evaluation of Synthetic and Generated Data

Synthetic data has mostly been judged in two ways: how statistically similar it is to the real data, and how well models trained on it perform. SynthTextEval [15] argues for a wider check that also includes distribution shifts, diversity, fairness, and privacy. Being as similar as possible to the reference corpus is not the same as being useful, since it leaves no room for new examples [10]. Alaa *et al.* proposed sample-level fidelity, diversity, and generalization metrics for generative models [16]. These metrics were designed to judge generated outputs, whereas we want to audit a labeled training set.

C. Lexical, Distributional, and Semantic Similarity

TF-IDF remains a simple and interpretable representation for comparing lexical characteristics and for baseline classifiers [17], usually combined with cosine similarity [18]. The Jaccard coefficient [19] gives a bounded measure of set overlap for vocabularies and n-gram inventories, and Jensen–Shannon divergence is a symmetric, bounded way to compare probability distributions [20]. For meaning beyond exact word overlap, Sentence-BERT produces sentence embeddings that can be compared with cosine similarity [21].

D. Diversity, Repetition, and Memorization

Lee *et al.* showed that deduplicating training data improves language models and reduces memorization [22], and Carlini *et al.* showed that models can reproduce verbatim training sequences [9]. Generated datasets are also prone to repetition and attribute bias [11], so we treat diversity as its own dimension.

E. Research Gap

Most of the generation papers above check only task accuracy, and the evaluation papers usually propose one metric at a time. We did not build a new generator. What we built is the workflow around evaluation: statistics, similarity, diversity checks, and a controlled utility test, all run from one application and saved so they can be repeated.

III. System Architecture

Fig. 1 shows the overall design. The user uploads two CSV files (original and synthetic) in the browser. The FastAPI backend checks the files, finds the text and label columns, runs the evaluation, and saves everything to PostgreSQL: dataset details, metric values, comparison records, and the settings used.

Inside the backend, work is split into dataset processing, linguistic evaluation, similarity and diversity analysis, and utility evaluation. A comparison step then lines up the original and synthetic results metric by metric, and the reporting layer turns them into the dashboard and the CSV, JSON, and PDF exports.

A. Detailed Explanation of the Architecture Diagram

The diagram has three rows, and the arrows show which way data moves.

Tier 1 – Interaction and orchestration. *User/Researcher* is the person running an evaluation. They supply two CSV files and pick a task (currently text classification). Everything they do goes through the *React Frontend*, which also draws the charts and starts exports. The frontend talks to the *FastAPI Backend* over REST; its endpoints handle uploads, new comparisons, comparison details, and report downloads. The backend calls the analysis engines in order and writes the results to *PostgreSQL*, including the full configuration, so an old experiment can be opened from the History page and run again.

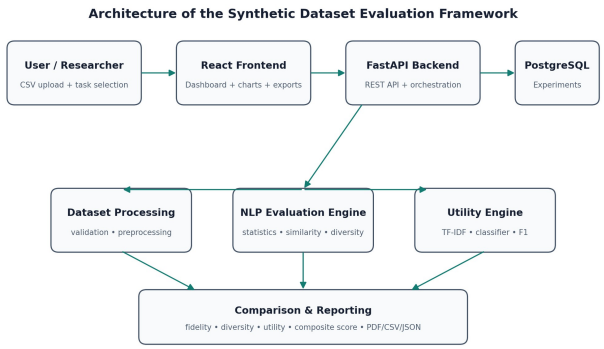


Fig. 1. System architecture of the proposed synthetic NLP dataset evaluation framework. The upper row shows the request path from the researcher to storage; the middle row shows the three analysis engines; the bottom block aggregates their outputs into scores and reports.

Tier 2 – Analysis engines. The arrow down from the backend splits three ways, because each comparison uses three components. *Dataset Processing* checks for missing values and duplicates, cleans whitespace, and tokenizes. The *NLP Evaluation Engine* in the middle computes statistics, similarity, and diversity. The *Utility Engine* builds TF-IDF features, trains the classifiers, and reports F1 scores.

Tier 3 – Aggregation and reporting. All three engines feed *Comparison & Reporting*. Here the results become original-vs-synthetic pairs, are grouped under fidelity, diversity, and utility, and are combined into the composite score.

We kept general text metrics apart from task-specific ones. Vocabulary or duplicate checks work on any text dataset, while the utility test depends on the task. For now that task is text classification, tested here on sentiment analysis.

IV. Methodology and Implementation

Notation: D_o and D_s denote the original and synthetic datasets, $n_o = |D_o|$ and $n_s = |D_s|$ their sizes, V_o and V_s their vocabularies, and $\epsilon = 10^{-9}$ a small constant that prevents division by zero. All similarity values lie in $[0, 1]$ and are multiplied by 100 when displayed as percentages.

A. Dataset Ingestion and Preprocessing

When a file is uploaded, the tool checks the CSV, counts missing values and duplicates, and asks the user to confirm which columns hold the text and the label. Both datasets then go through identical preprocessing. Semantic analysis uses the raw text; lexical statistics use the cleaned, tokenized version.

Normalization collapses consecutive whitespace and trims

each record. Tokenization is Unicode-aware: after lowercasing, a character is kept as part of a token when its Unicode general category is a letter (L), mark (M), or number (N). Retaining the *mark* category is essential for Devanagari, where vowel signs (matras) and the virama are combining marks; an ASCII-only tokenizer would split Hindi words into fragments. Stopwords are not removed by default so that informative task vocabulary is not discarded.

B. Dataset Statistics

For a dataset with documents $d_1, \ldots, d_n$, let ℓ_i be the number of tokens in d_i and $N = \sum_i \ell_i$ the total token count. The mean text length and the type-token ratio (TTR) are

$$\mu = \frac{1}{n}\sum_{i=1}^{n} \ell_i, \qquad \mathrm{TTR} = \frac{|V|}{N}. \tag{1}$$

Median, standard deviation, minimum, and maximum length, word-frequency tables, top unigrams and bigrams, and the class proportions $p(c) = n_c/n$ are also recorded.

For scalar statistics such as record count, mean length, vocabulary size, and TTR, the comparison layer converts the original value a and synthetic value b into a bounded similarity:

$$S_{\mathrm{rel}}(a, b) = \max\!\left(0,\ 1 - \frac{|a-b|}{\max(|a|, \epsilon)}\right). \tag{2}$$

C. Fidelity Metrics

1) Vocabulary and n-gram overlap. Vocabulary overlap is based on set intersection and union using the Jaccard coefficient [19]:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}. \tag{3}$$

It is applied to the vocabularies ($S_{\mathrm{voc}} = J(V_o, V_s)$), to unigram sets ($S_{1g}$), and to the sets of distinct bigrams ($S_{2g} = J(G_o^{(2)}, G_s^{(2)})$).

2) TF-IDF similarity. A single vectorizer (unigrams and bigrams, up to 10,000 features) is fitted on the union of both corpora so that both share one feature space. The weight of term t in document d is

$$w_{t,d} = \mathrm{tf}_{t,d} \cdot \left(\ln \frac{1+n}{1+\mathrm{df}_t} + 1\right), \tag{4}$$

followed by L_2 normalization of each document vector [17, 18]. The dataset centroids $\bar{\mathbf{x}}_o$ and $\bar{\mathbf{x}}_s$ are the means of the document vectors, and the similarity is their cosine:

$$S_{\mathrm{tfidf}} = \cos(\bar{\mathbf{x}}_o, \bar{\mathbf{x}}_s) = \frac{\bar{\mathbf{x}}_o \cdot \bar{\mathbf{x}}_s}{\|\bar{\mathbf{x}}_o\|\,\|\bar{\mathbf{x}}_s\|}. \tag{5}$$

3) Semantic similarity. A random sample (fixed seed) of each dataset is encoded with a sentence-embedding model [21] into L_2-normalized vectors $\mathbf{e}_i$. The semantic similarity is the cosine between the embedding centroids,

$$S_{\mathrm{sem}} = \cos\!\Big(\tfrac{1}{m_o}\textstyle\sum_i \mathbf{e}_i^{o},\ \tfrac{1}{m_s}\sum_j \mathbf{e}_j^{s}\Big). \tag{6}$$

If the embedding model is unavailable, the system falls back to the TF-IDF centroid similarity and records that fallback in the experiment configuration.

4) Class-distribution similarity. Let P and Q be the original and synthetic label distributions and $M = \frac{1}{2}(P+Q)$. The Jensen–Shannon divergence [20] with base-2 logarithms is

$$\mathrm{JSD}(P\|Q) = \tfrac{1}{2}\mathrm{KL}(P\|M) + \tfrac{1}{2}\mathrm{KL}(Q\|M), \tag{7}$$

where $\mathrm{KL}(P\|M) = \sum_c P(c)\log_2 \frac{P(c)}{M(c)}$. The divergence is bounded in $[0, 1]$ and is converted to a similarity through the Jensen–Shannon distance:

$$S_{\mathrm{cls}} = 1 - \sqrt{\mathrm{JSD}(P\|Q)}. \tag{8}$$

5) Fidelity score. The fidelity score is the mean of seven bounded components:

$$F = \frac{100}{7}(S_{\mathrm{voc}} + S_{1g} + S_{2g} + S_{\mathrm{tfidf}} + S_{\mathrm{sem}} + S_{\mathrm{cls}} + S_{\mathrm{len}}), \tag{9}$$

where $S_{\mathrm{len}} = S_{\mathrm{rel}}(\mu_o, \mu_s)$ from (2).

D. Diversity Metrics

A dataset can be faithful to the original and still have little variety, so diversity gets its own score. We measure the unique-text ratio, duplicate rate, vocabulary richness, and phrase repetition:

$$U = \frac{|\{\text{distinct texts}\}|}{n}, \qquad R_{\mathrm{dup}} = \frac{n_{\mathrm{dup}}}{n}, \tag{10}$$

where n_{dup} counts records whose normalized text already occurred earlier in the dataset. Phrase repetition is measured with the repeated trigram rate

$$R_{3g} = \frac{\sum_{g:\, c(g)>1} c(g)}{\sum_g c(g)}, \tag{11}$$

where $c(g)$ is the frequency of trigram g. The diversity score averages five components:

$$\begin{aligned} D = \frac{100}{5}(&U_s + (1 - R_{\mathrm{dup},s}) + \mathrm{TTR}_s \\ &+ S_{\mathrm{rel}}(U_o, U_s) + S_{\mathrm{rel}}(R_{3g,o}, R_{3g,s})). \end{aligned} \tag{12}$$

A synthetic corpus containing many repeated templates can therefore receive a low diversity score even if its class distribution is correct.

E. Downstream Utility Experiment

Fig. 2 shows the utility test. We train a classifier on the real training split and test it on the real held-out split. We then train the same kind of model on the synthetic data and test it on that same real split. Comparing the two tells us how well the synthetic data can replace the real data for this task. Features are TF-IDF vectors, the models are standard linear classifiers, and we record accuracy, precision, recall, macro-F1, and weighted-F1.

In Fig. 2, the upper path is the Real→Real baseline and the lower path is the Synthetic→Real experiment. The dashed arrow

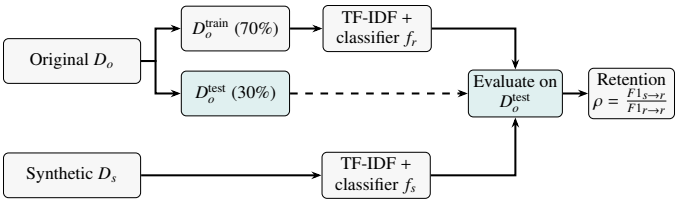


Fig. 2. Controlled utility protocol. The original data are split once (stratified, fixed seed); the real-trained model f_r and the synthetic-trained model f_s use the same vectorizer and classifier family and are both scored on the identical real test split, which is never used for training.

marks the 30% real test split, which is only ever used for testing. Both paths use the same three classifiers: logistic regression with balanced class weights, a linear support vector machine, and multinomial naive Bayes [18]. For each class c with precision P_c and recall R_c,

$$F1_c = \frac{2P_cR_c}{P_c + R_c}, \qquad F1_{\text{macro}} = \frac{1}{K}\sum_{c=1}^{K} F1_c, \tag{13}$$

where K is the number of classes [23]. For each model family k, the macro-F1 retention is

$$\rho_k = \min\left(\frac{F1^{(k)}_{\text{macro}}(s\to r)}{\max\left(F1^{(k)}_{\text{macro}}(r\to r), \epsilon\right)},\ 1.2\right), \tag{14}$$

and the utility score uses the best model, $U_{\text{util}} = 100 \cdot \min(1, \max_k \rho_k)$. Utility retention may exceed 100% when synthetic-trained performance is higher than the real-data baseline; the dashboard normalizes the dimension score to a maximum of 100 while retaining the raw retention value for interpretation.

F. Composite Score

The composite score is a weighted sum with 40% for fidelity, 30% for diversity, and 30% for utility:

$$S_{\text{overall}} = w_F F + w_D D + w_U U_{\text{util}}, \tag{15}$$

with $(w_F, w_D, w_U) = (0.4, 0.3, 0.3)$. If a user enters other weights, they are rescaled to add up to one. These weights are our own choice, not an accepted standard. A score of 80 or above counts as strong, 60–79 as moderate, and anything below 60 is shown on the dashboard as "Needs Review."

G. Implementation

The dashboard walks the user through five steps: dataset type, original dataset, synthetic dataset, evaluation, and results. It is written in React and TypeScript with Vite, styled with Tailwind CSS, and uses TanStack Query for server state and Recharts for charts.

On the server side we used FastAPI and Pydantic for typed requests and responses, SQLAlchemy for the database, pandas and NumPy for data handling, SciPy for Jensen–Shannon divergence, scikit-learn [18] for vectorizing and classification, and ReportLab for PDF reports. Experiments are stored in PostgreSQL; SQLite works for local testing. Each run saves its

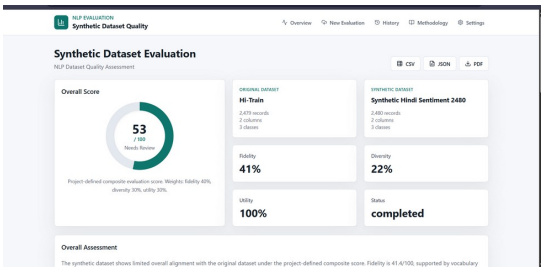


Fig. 3. Evaluation dashboard showing the overall score and the three evaluation dimensions.

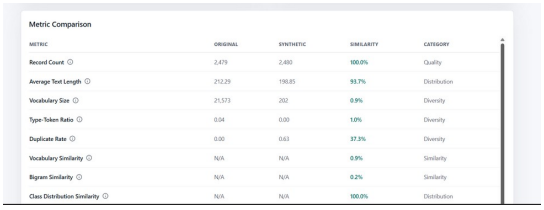


Fig. 4. Metric-level comparison between the original and synthetic Hindi sentiment datasets.

model names, random seed, sample size, preprocessing settings, metric definitions, and weights, so the same run can be repeated later.

The tool does not generate synthetic data. It takes the synthetic dataset as an input, which means it can compare datasets made by any method without being tied to one generator.

V. Dataset Description and Experimental Setup

For testing we used Hindi sentiment data with three labels (negative, neutral, positive). We compared the original corpus (labeled *Hi-Train* in the dashboard) with a synthetic corpus of approximately equal size (*Synthetic Hindi Sentiment 2480*). Each file has two columns, text and label. As Fig. 5 shows, the label shares are roughly 30% negative, 28% neutral, and 42% positive in both corpora.

All metrics used the configuration in Section IV, with a stratified 70/30 split of the original data, a fixed random seed, and weights $(0.4, 0.3, 0.3)$.

VI. Results and Discussion

A. Description of the Dashboard Screens

Evaluation dashboard (Fig. 3). Along the top are the navigation links (Overview, New Evaluation, History, Methodology, Settings) and buttons to export CSV, JSON, or PDF. On the left, a donut gauge shows the overall score of 53/100 with the label "Needs Review". The two cards on the right name the datasets, and the four cards below them show fidelity 41%, diversity 22%, utility 100%, and the run status (*completed*).

Metric comparison table (Fig. 4). Each row is one metric: its original value, synthetic value, similarity, and category. Record count and text length have a value for each dataset. Overlap measures such as vocabulary and bigram similarity only exist as a pair, so their value columns show "N/A".

TABLE I
SUMMARY OF EXPERIMENTAL RESULTS

Metric	Original vs. Synthetic / Result
Record count	2,479 vs. 2,480 (100.0%)
Mean text length	212.29 vs. 198.85 (93.7%)
Vocabulary size	21,573 vs. 202 (0.9%)
Type-token ratio	0.04 vs. approx. 0.00 (1.0%)
Duplicate rate	approx. 0.00 vs. 0.63 (37.3%)
Bigram similarity	0.2%
Class-distribution similarity	100.0%
Fidelity score	41%
Diversity score	22%
Utility score	100% normalized; raw macro-F1 retention approx. 120%
Overall composite	53 / 100

Assessment and charts (Fig. 5). The *Overall Assessment* box holds the text summary the tool writes for each run. Under it, the *Class Distribution* chart puts the original (teal) and synthetic (orange) label shares side by side, and the bars are almost the same height. The *Dimension Scores* chart plots the three scores out of 100.

B. Quantitative Results

The results show a clear separation between the evaluation dimensions. The average text length was 212.29 for the original data and 198.85 for the synthetic data, producing a reported similarity of 93.7%. Class-distribution similarity was reported as 100.0%, indicating that the synthetic data reproduced the label proportions closely.

However, lexical diversity was substantially weaker. The original vocabulary contained 21,573 unique tokens compared with 202 in the synthetic dataset. The resulting reported vocabulary similarity was only 0.9%, while bigram similarity was 0.2%. TTR was 0.04 for the original dataset and approximately 0.00 for the synthetic dataset. The duplicate rate increased from approximately 0.00 in the original dataset to 0.63 in the synthetic dataset, corresponding to a low duplicate-rate similarity of 37.3%; in other words, only about 37% of the synthetic records are distinct texts. These results indicate substantial repetition and limited lexical novelty in the synthetic corpus.

The dashboard reported a project-defined fidelity score of 41% (41.4), diversity of 22% (22.42), and a normalized utility score of 100%, producing an overall composite score of 53/100. Substituting into (15) gives $0.4(41.4) + 0.3(22.42) + 0.3(100) = 53.3$, which is displayed as 53. The raw downstream macro-F1 retention was approximately 120%. Thus, the synthetic dataset performed strongly in the selected downstream experiment despite exhibiting weak lexical fidelity and diversity. This is an important result: task utility alone would have overstated the quality of the synthetic corpus.

C. Discussion

The results show clearly that one number is not enough. Class balance and utility were excellent, while vocabulary overlap and diversity were poor. Table I lists the values.

The gap between utility and diversity deserves attention. A classifier can learn to separate positive, neutral, and negative text from repeated, template-like examples, even if those examples use only a tiny part of the original vocabulary. So the 120% macro-F1 retention does not mean the synthetic data is better than the real data. It only means the synthetic examples were enough for this task under this setup.

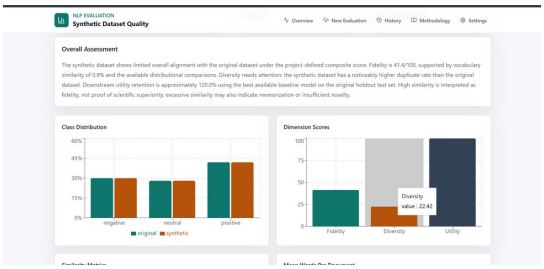


Fig. 5. Overall assessment and dimension scores generated by the evaluation framework.

Looking inside the scores explains the 53. In fidelity (9), class similarity and length similarity are close to one, but the three overlap scores are close to zero, which brings the average down to 41.4. In diversity (12), the unique-text ratio and $(1 - R_{\text{dup}})$ are both about 0.37 and the synthetic TTR is almost zero, giving 22.42. Utility is the only dimension that reaches the maximum.

VII. LIMITATIONS

This study has clear limits. We tested one Hindi sentiment dataset against one synthetic dataset, so the results do not carry over to every NLP task. The lexical metrics depend on how the text is tokenized and preprocessed. Semantic similarity depends on which embedding model is used, and no model captures every linguistic or cultural detail of Hindi.

The composite weights are our own choice. They make the dashboard easy to read, but another project could reasonably weight fidelity, diversity, and utility differently. Finally, good utility says nothing about privacy, and duplicate detection only hints at overlap; it is not a proper privacy-leakage test.

VIII. CONCLUSION AND FUTURE WORK

We built a tool that judges a synthetic NLP dataset on fidelity, diversity, and downstream utility, instead of on accuracy alone. Users upload a real and a synthetic dataset, and the tool computes matched metrics, runs a controlled classification test, and produces a report they can export.

On the Hindi sentiment data, the synthetic set matched the class balance and trained a strong classifier. It also had only 202 distinct words against 21,573 in the original, and 63% of its rows were duplicates. A report based on accuracy would have missed both issues; the score of 53/100 reflects them.

Next, we plan to add more tasks (topic classification, spam detection, named-entity recognition) and more languages. Other useful additions are stronger privacy checks, fairness evaluation, human ratings, confidence intervals over repeated runs, and comparisons between several generation methods. A larger benchmark across languages and domains would make the con-

clusions stronger.

References

[1] A. Vaswani *et al.*, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2017, pp. 5998–6008.

[2] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.

[3] T. B. Brown *et al.*, "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 1877–1901.

[4] T. Schick and H. Schütze, "Generating datasets with pretrained language models," in *Proc. EMNLP*, 2021, pp. 6943–6951.

[5] Y. Meng, J. Huang, Y. Zhang, and J. Han, "Generating training data with language models: Towards zero-shot language understanding," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 35, 2022.

[6] Y. Wang *et al.*, "Self-Instruct: Aligning language models with self-generated instructions," in *Proc. ACL*, 2023, pp. 13484–13508.

[7] J. Wei and K. Zou, "EDA: Easy data augmentation techniques for boosting performance on text classification tasks," in *Proc. EMNLP-IJCNLP*, 2019, pp. 6382–6388.

[8] S. Y. Feng *et al.*, "A survey of data augmentation approaches for NLP," in *Findings of ACL-IJCNLP*, 2021, pp. 968–988.

[9] N. Carlini *et al.*, "Extracting training data from large language models," in *Proc. 30th USENIX Security Symp.*, 2021, pp. 2633–2650.

[10] A. Havrilla *et al.*, "Surveying the effects of quality, diversity, and complexity in synthetic data from large language models," arXiv preprint arXiv:2412.02980, 2024.

[11] Y. Yu *et al.*, "Large language model as attributed training data generator: A tale of diversity and bias," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS) Datasets and Benchmarks Track*, vol. 36, 2023.

[12] P. Joshi, S. Santy, A. Budhiraja, K. Bali, and M. Choudhury, "The state and fate of linguistic diversity and inclusion in the NLP world," in *Proc. ACL*, 2020, pp. 6282–6293.

[13] D. Kakwani *et al.*, "IndicNLPSuite: Monolingual corpora, evaluation benchmarks and pre-trained multilingual language models for Indian languages," in *Findings of EMNLP*, 2020, pp. 4948–4961.

[14] M. Doshi and P. Bhattacharyya, "Synthetic data for multilingual NLP: A survey," CFILT, Indian Institute of Technology Bombay, 2024.

[15] K. Ramesh *et al.*, "SynthTextEval: Synthetic text data generation and evaluation for high-stakes domains," arXiv preprint arXiv:2507.07229, 2025.

[16] A. Alaa, B. van Breugel, E. S. Saveliev, and M. van der Schaar, "How faithful is your synthetic data? Sample-level metrics for evaluating and auditing generative models," in *Proc. 39th Int. Conf. Mach. Learn. (ICML)*, PMLR vol. 162, 2022, pp. 290–306.

[17] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Inf. Process. Manage.*, vol. 24, no. 5, pp. 513–523, 1988.

[18] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[19] P. Jaccard, "The distribution of the flora in the alpine zone," *New Phytologist*, vol. 11, no. 2, pp. 37–50, 1912.

[20] J. Lin, "Divergence measures based on the Shannon entropy," *IEEE Trans. Inf. Theory*, vol. 37, no. 1, pp. 145–151, 1991.

[21] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. EMNLP-IJCNLP*, 2019, pp. 3982–3992.

[22] K. Lee *et al.*, "Deduplicating training data makes language models better," in *Proc. ACL*, 2022, pp. 8424–8445.

[23] M. Sokolova and G. Lapalme, "A systematic analysis of performance measures for classification tasks," *Inf. Process. Manage.*, vol. 45, no. 4, pp. 427–437, 2009.